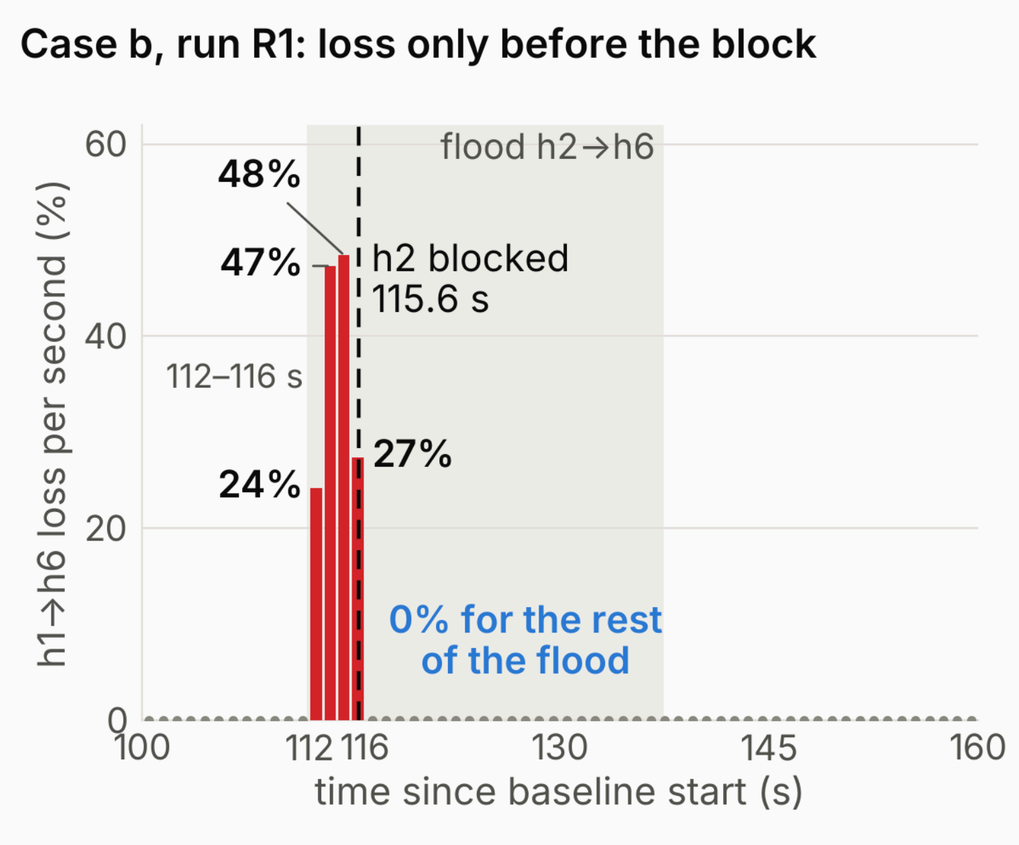
CEG5101 CA2 · GROUP 5

From 48% Loss to Zero

An Anomaly-Aware Routing and Blocking Controller · CEG5101 CA2

48% → 0%

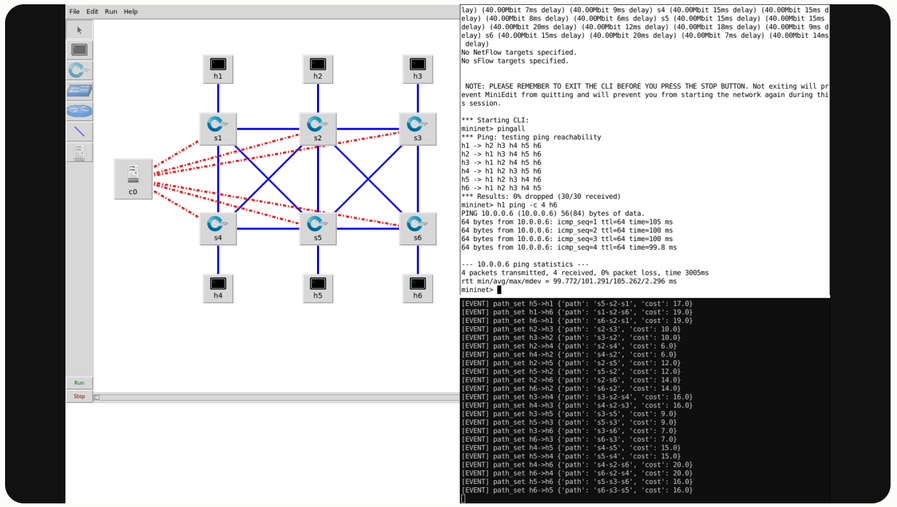
h1→h6 loss: worst second before detection vs after the block (R1, case b)



Group 5 · Yang Kaiming (A0326309E) · Chen Danze (A0328226E)

12 Nov 2026 · 10:25–10:50 · SDE1-SR1 · Ryu 4.34 · OpenFlow 1.3 · Mininet

Task 1: One Mesh, One Flow to Protect, Three Attacks

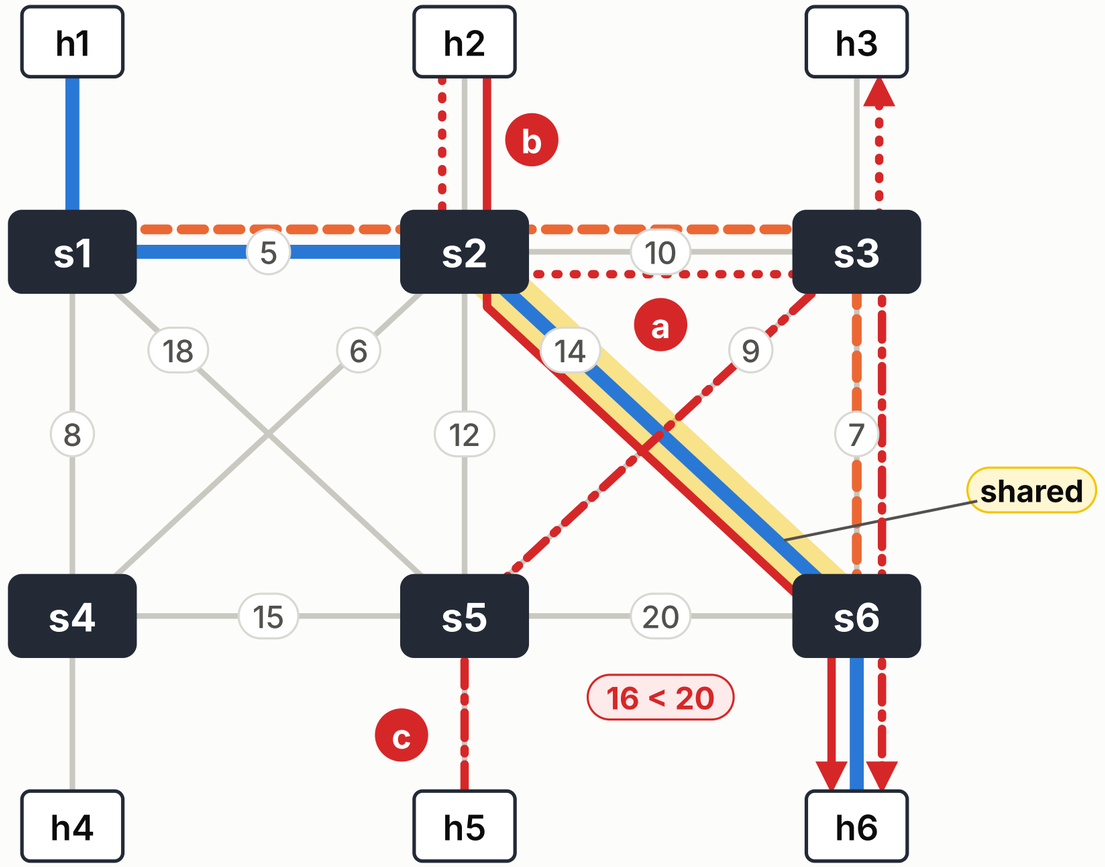


MiniEdit in the Mininet 2.3.0 VM · OpenFlow 1.3 · pingall 0% dropped

6 switches · host h*i* on s*i*

40 Mbps links · C0 = link delay (ms)

- h1→h6 s1-s2-s6 = 19
- backup s1-s2-s3-s6 = 22
- attacks a · b · c



Running example: case b (s2–s6)

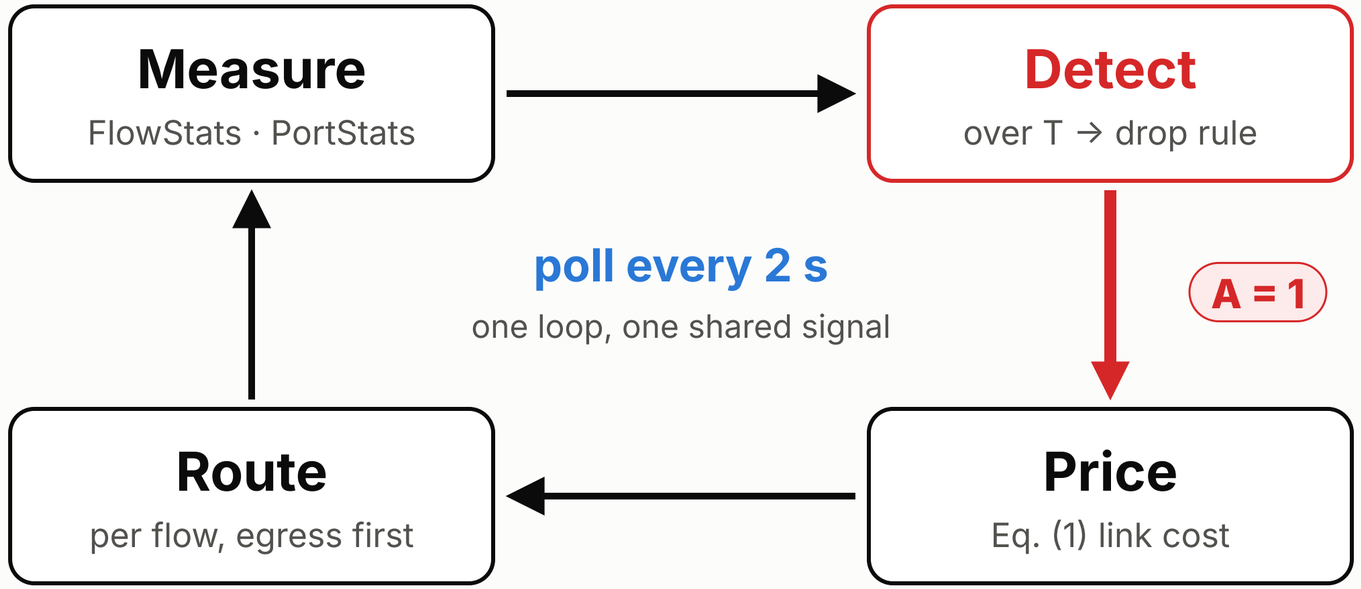
Two Reference Controllers, Two Blind Spots, One 2-Second Loop

`resilient_controller`
routes, blind to attacks
blind spot

`anomaly_detection`
blocks, never reroutes
blind spot

→

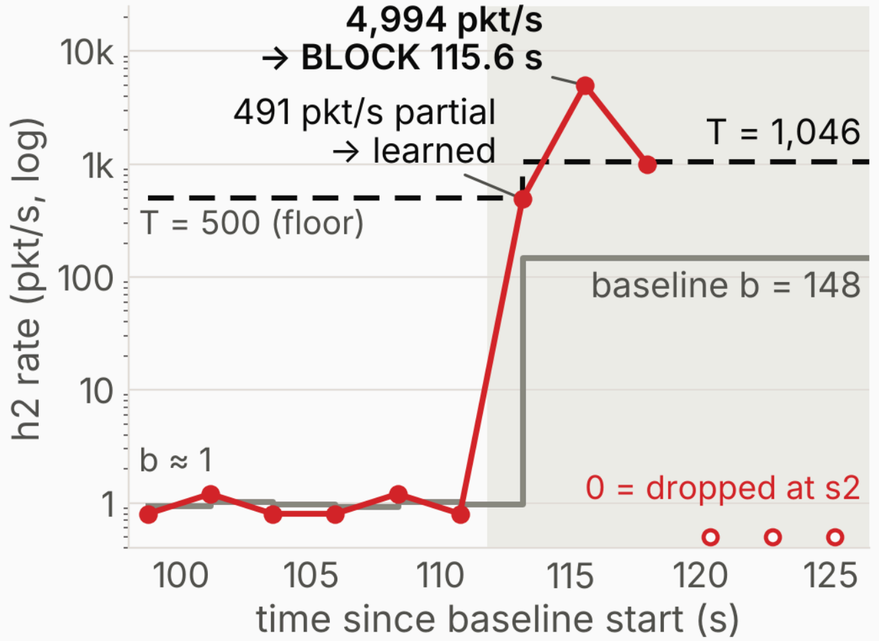
`integrated_controller`
one loop, both jobs



Task 2: Catch the Attacker, Not the Victim

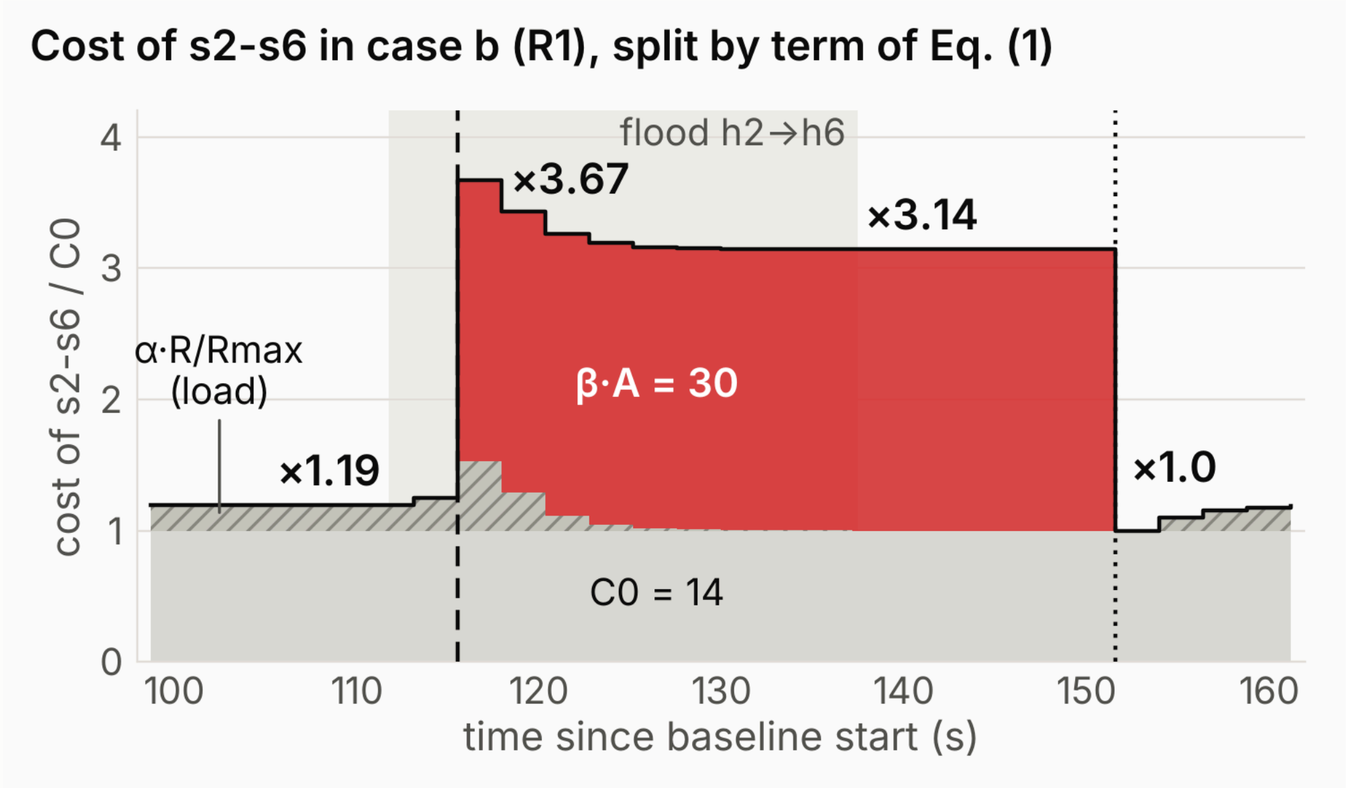
NAIVE	FAILS BECAUSE	OUR RULE
Count echo replies	Victim h6 gets blocked	**Skip ICMP echo replies** D4
Update EMA, then compare	≈21× threshold, baseline poisoned	**Compare first, never learn anomalies** D5
Fixed 3× multiple	Ignores how bursty sources are	**max(3b, b+4σ, 500) + warm-up** D6

Detector on h2 during case b (R1)



Task 3: Price the Attack into the Link

$$C_{ij}(t) = \underbrace{C0}_{\text{static: link delay (ms)}} + \underbrace{\alpha \cdot R/R_{max}}_{\substack{\text{load weight } \alpha = 10: \\ \text{fast, fades after the block}}} + \underbrace{\beta \cdot A}_{\substack{\text{anomaly weight } \beta = 30: \\ \text{memory while the attack lasts}}}$$



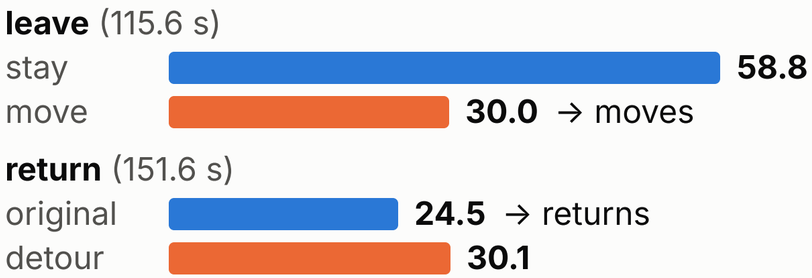
β = 30 ≫ **≈7.7 needed**
detour 3 + own load ≈2.7 + margin 2

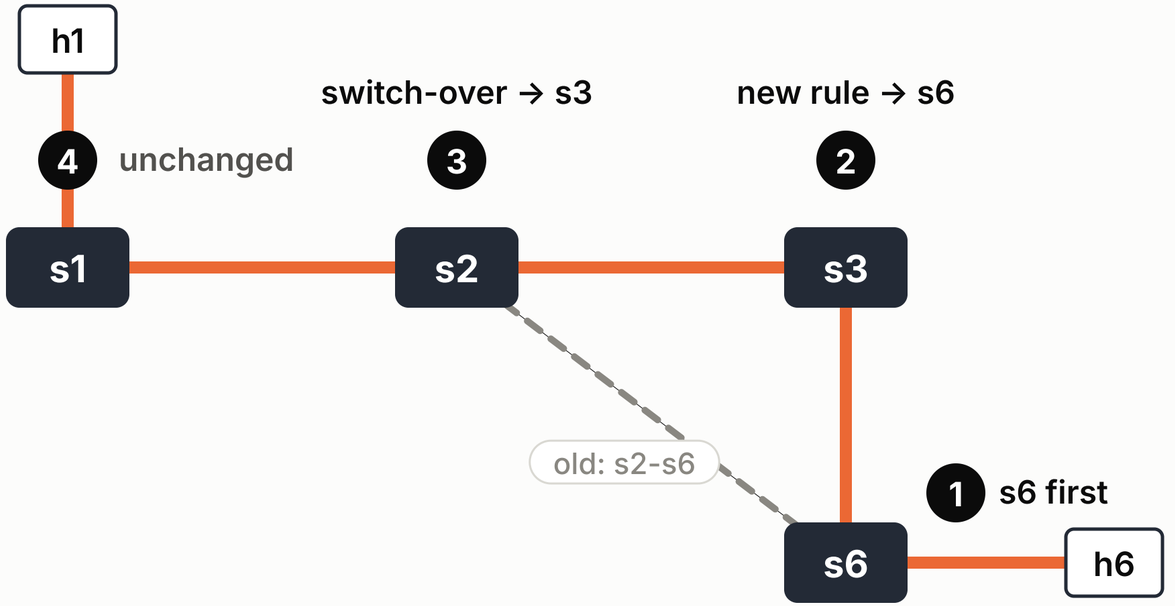
Prediction: plateau (C0 + 30) / C0

LINK	CASE	C0	PLATEAU
s2-s3	a	10	**4.0**
s2-s6	b	14	**3.14**
s3-s5	c	9	**4.33**
s3-s6	c	7	**5.29**

Task 4: Move Only When Clearly Cheaper, Install Egress First

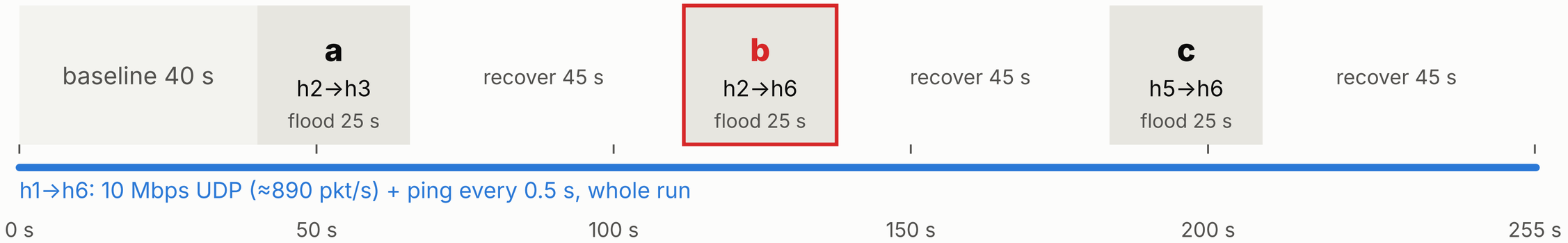
1. **What-if:** add own load D9
2. **Move only if ≥ 2 cheaper**
3. **Egress first**, then delete stale D10





Same rule: A = 1 → leave · A → 0 → return · link down → cost ∞

Experiment: One Timeline, Three Cases, Three Runs



Measured independently of the controller

iperf at h6
loss, throughput per second
`iperf_server.txt`

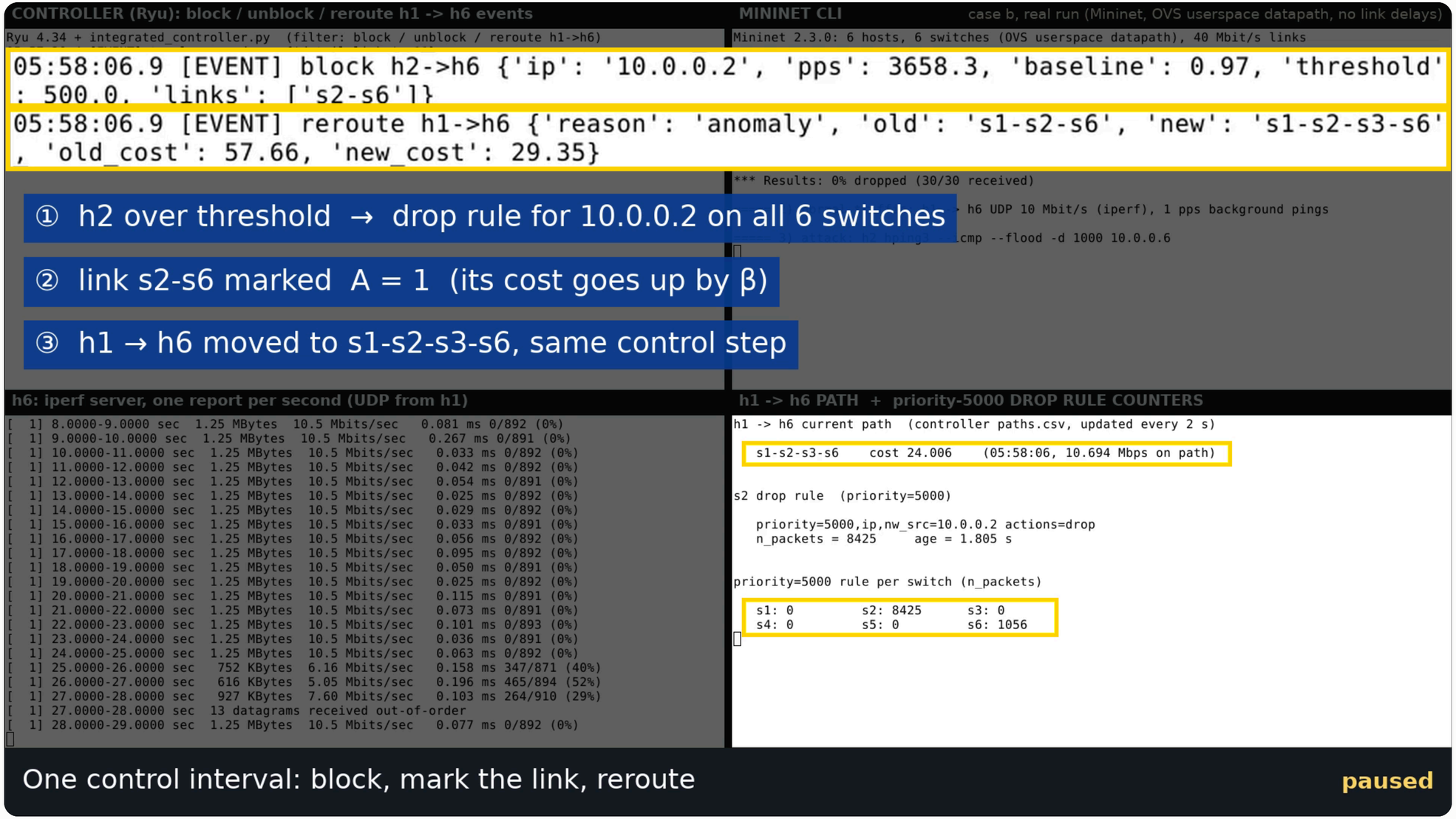
tcpdump at the victim
attack packets that got through
`victim_capture_*.txt`

Controller logs
events, link costs, paths
`controller/*.csv`

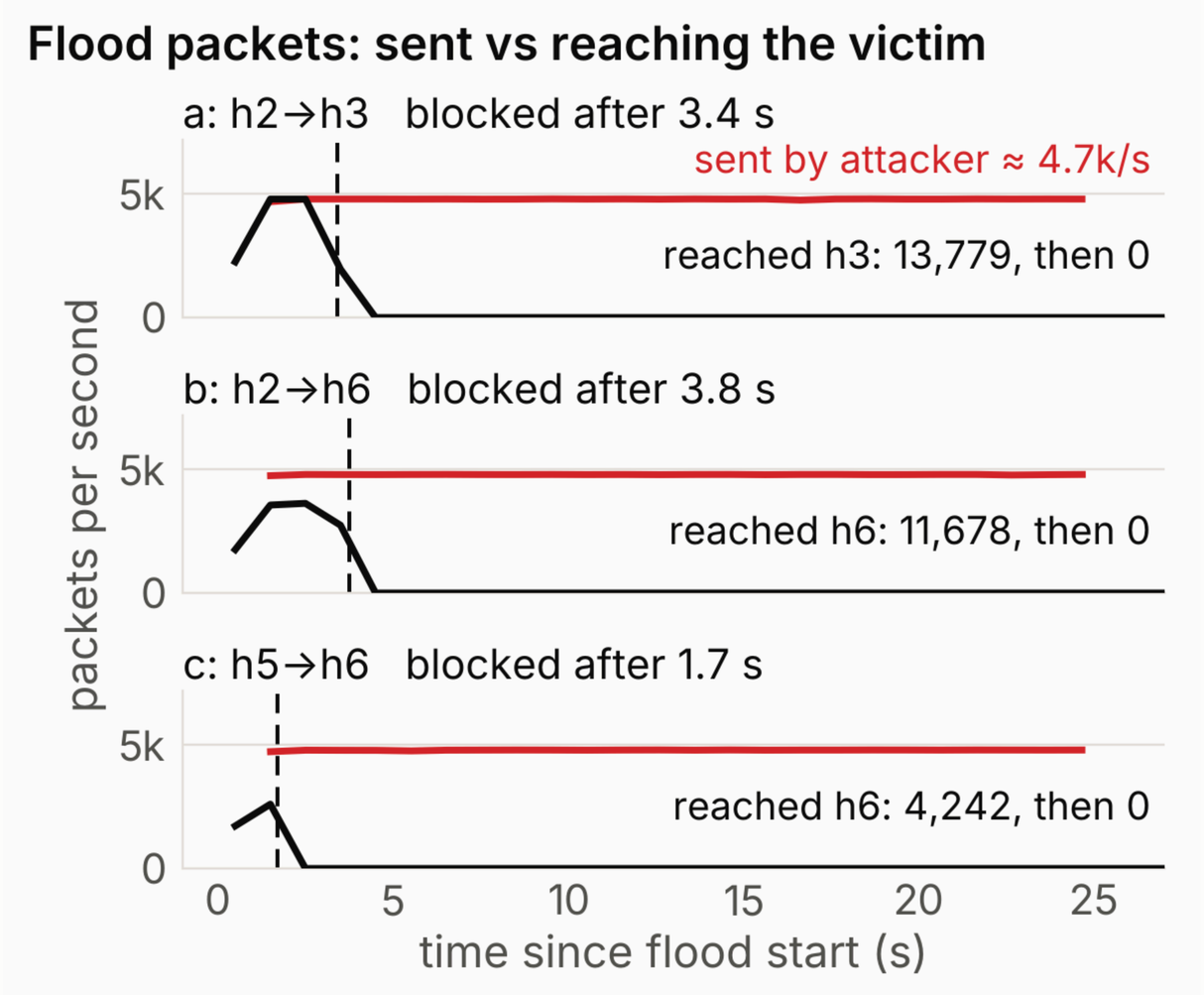
Flow-table dumps
drop-rule counters
`flows/*.txt`

R1–R3 · container: userspace OVS, no link delay

Demo: Case b in a Real Run



Detection and Blocking Results: One or Two Polls, Then Nothing Gets Through



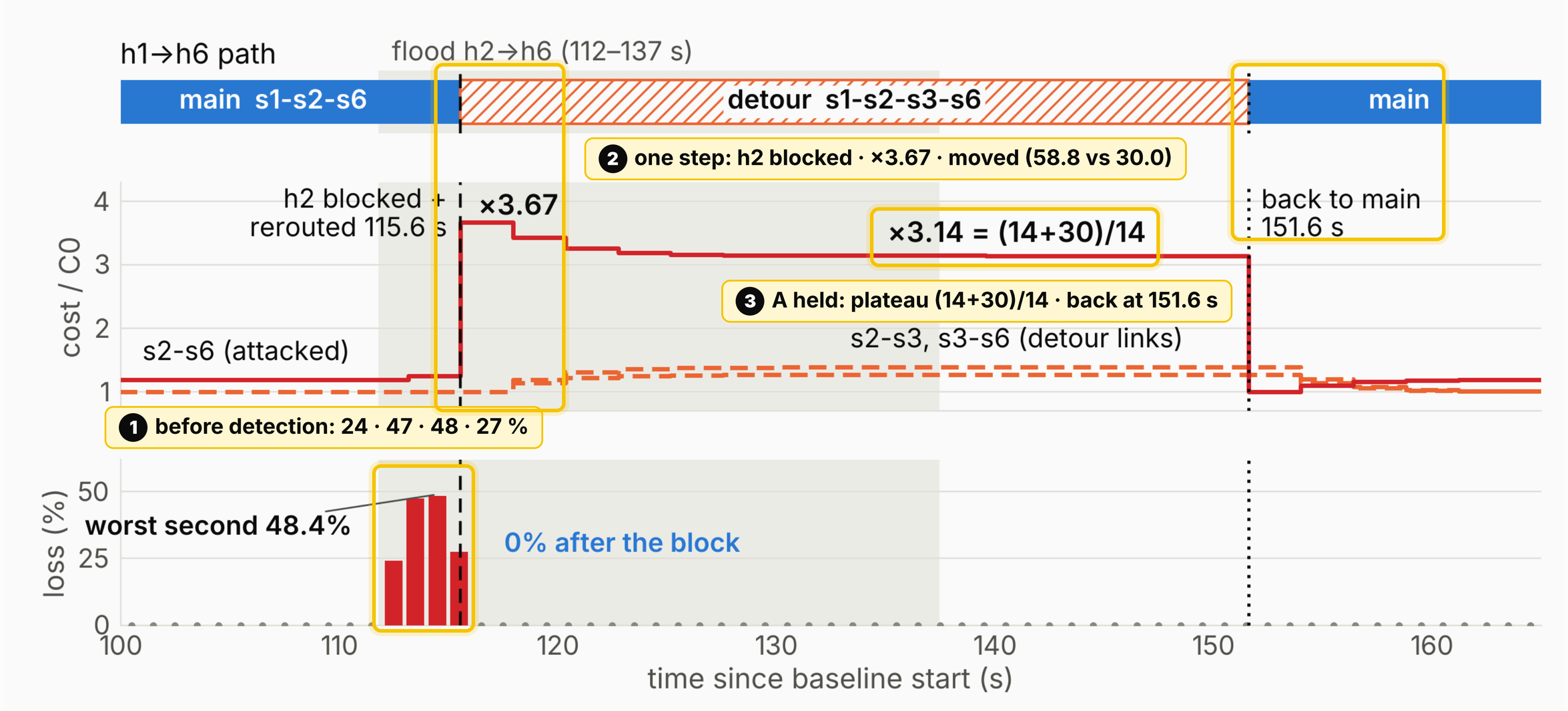
Detection delay (flood start → block)

CASE	R1	R2	R3
a	3.4 s	2.2 s	3.5 s
b	**3.8 s**	2.6 s	**1.5 s**
c	1.7 s	3.0 s	1.8 s

0 attack packets later than 1 s after the block, in all nine floods.

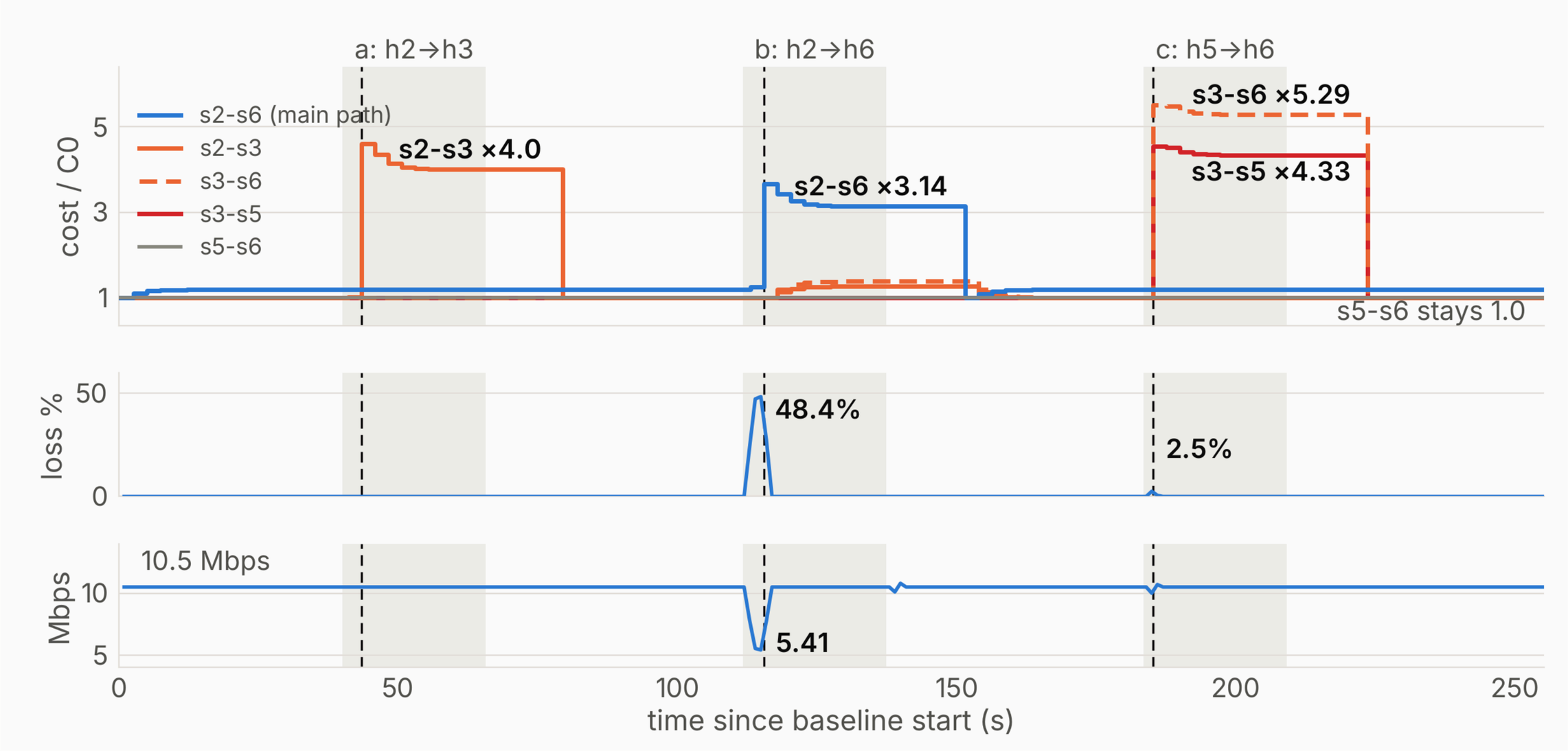
Every lost h1→h6 packet fell inside this detection delay.

Case b: Block and Reroute in One Control Interval



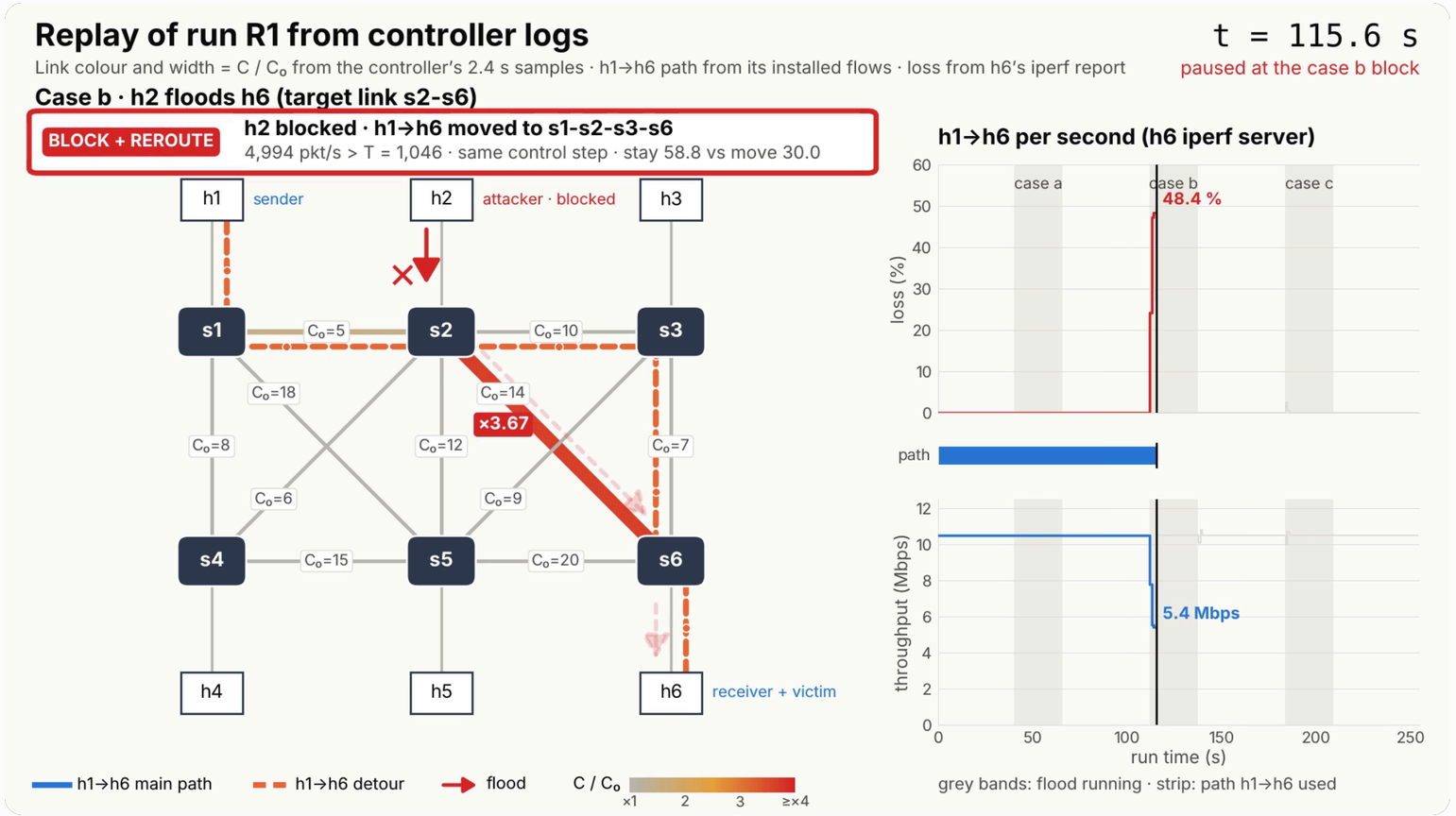
back ≈14 s after the flood

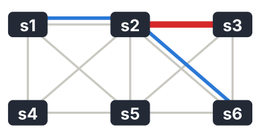
Task 5: Cost, Loss and Throughput over the Whole Run



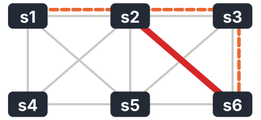
s5-s6 stays at 1.0: never used → next slide

Cases a and c: Not Rerouting Is the Correct Answer

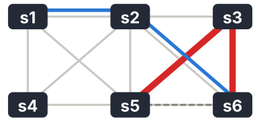




a · h2→h3
s2-s3 ×4.0
h1→h6 stays
loss 0%



b · h2→h6
s2-s6 ×3.14
moves, returns
worst second 34–57%



c · h5→h6
s3-s5 ×4.33
s3-s6 ×5.29
stays
worst second 2.5–6.1%

h1→h6 detour marked A = 1

A lands only where the attack went; a and c never touch h1→h6.

What Each Part Buys Us: Beta Off, Link Down

Case b with a smaller anomaly weight β

β	S2-S6 COST	H1→H6	WORST 1 s	AFTER BLOCK
0	1.20×	stayed	30.4%	0%
2	1.34×	stayed	30.1%	0%
30	**3.15–3.16×***	**moved**	34.4–56.6%	0%

* mean from 3 s after the block to the flood end (R1–R3); the plateau converges to (14+30)/14 = 3.14

s2-s6 cut for 20 s

7 ms

to reroute · **11 / 71,515** lost
back 0.7 s after restore

h1→h6 path over the 80 s run



Bug found: ready flag dropped on link-down → ARP ignored; fixed by latching ready (D13)

Where the Loss Lives: Limitations and Takeaways

h1→h6 during a case b flood



LIMITATION	NEXT STEP
Detection delay 1.5–3.8 s	1 s polls or sFlow
Spoofed sources evade IP block	reroute + per-destination meters

1.5–3.8 s
to block, all nine floods

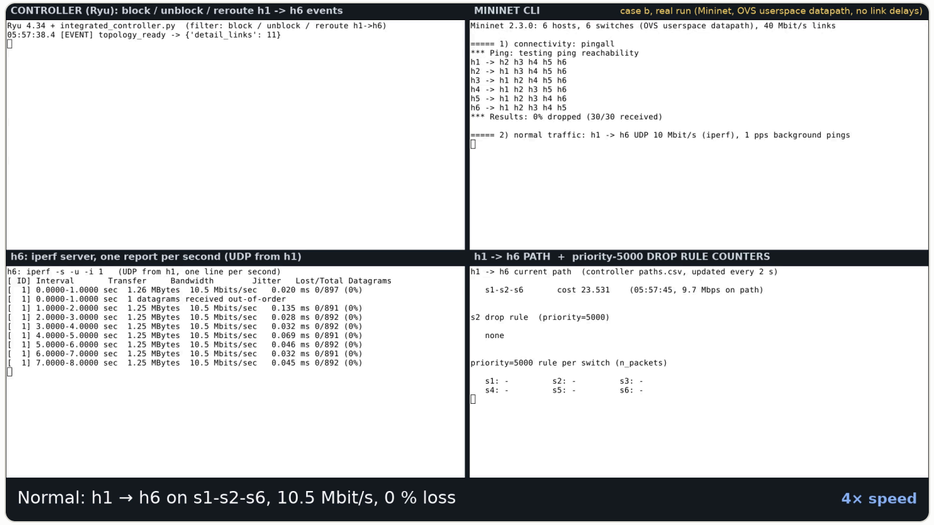
0
attack packets after block + 1 s

2 reroutes
away and back

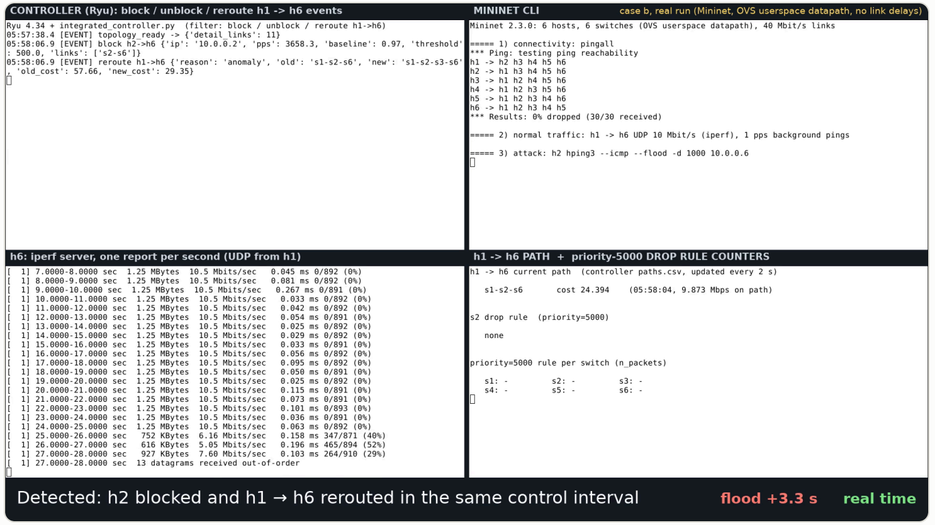
Questions?

Contributions & tools: slide 23

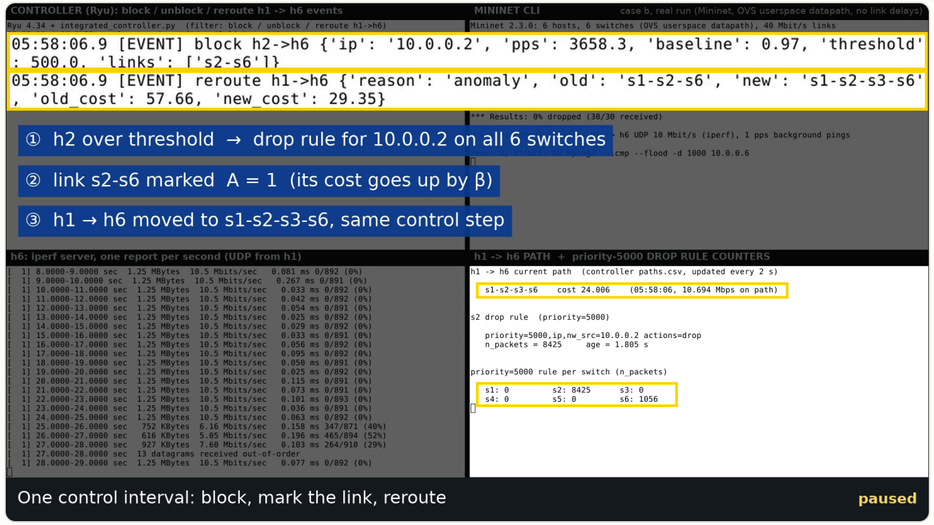
Case b Demo: Four Stills



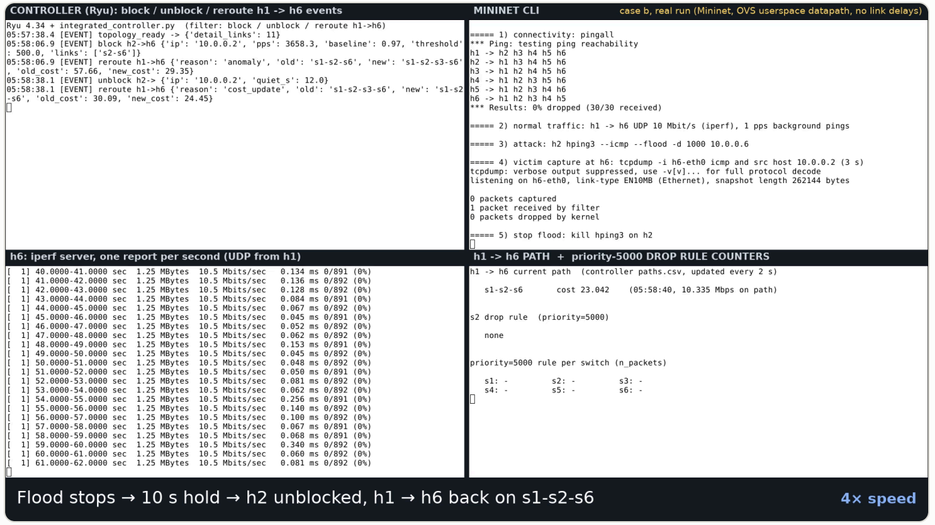
1 Normal (4×): s1-s2-s6, 0% loss



2 Block + reroute in one control step

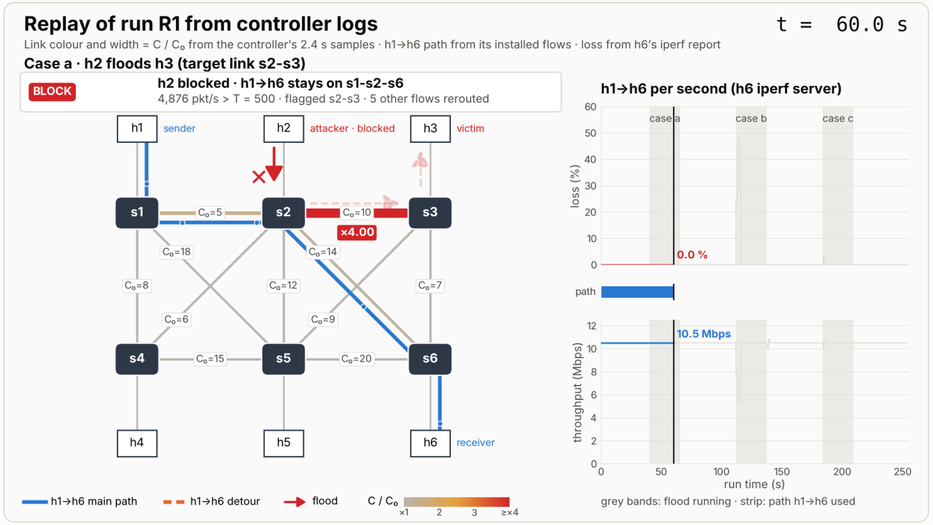


3 Paused: drop on 6 switches · A = 1 · detour

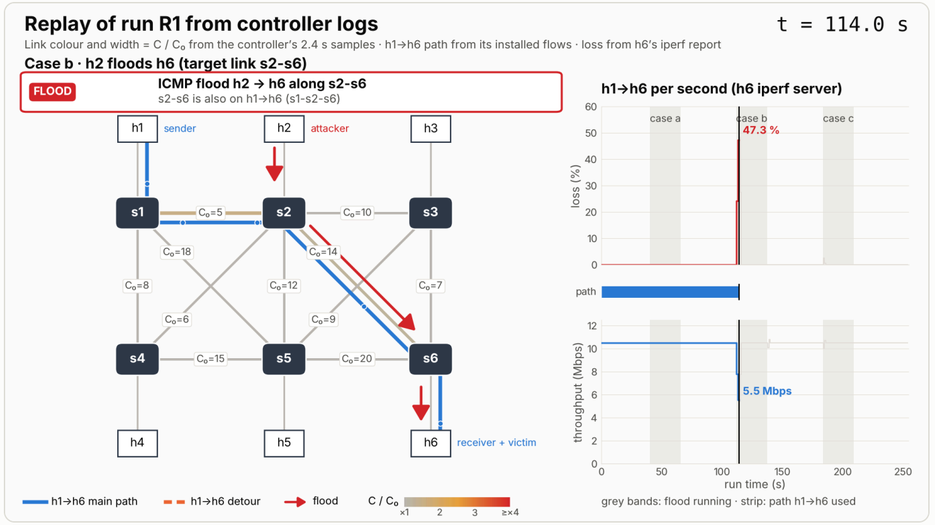


4 Back on s1-s2-s6 · tcpdump: 0 packets

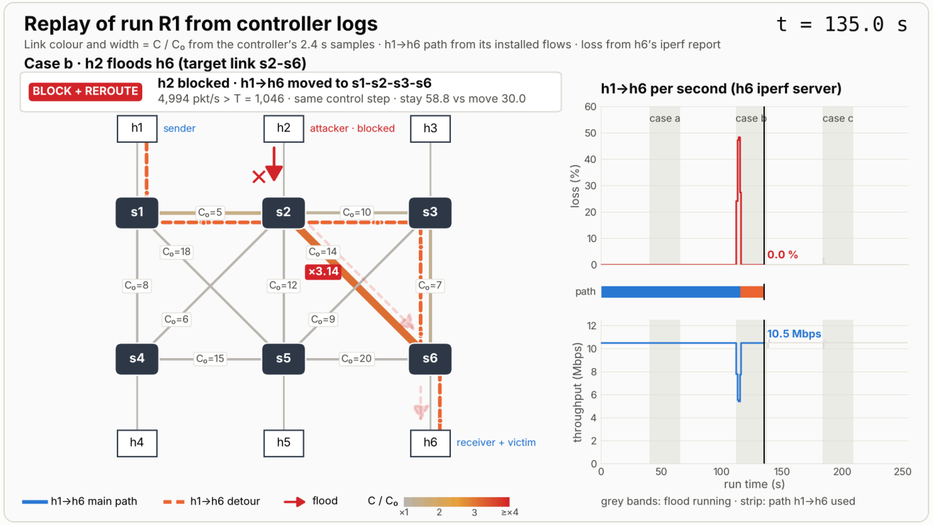
Replay of Run R1: Four Stills



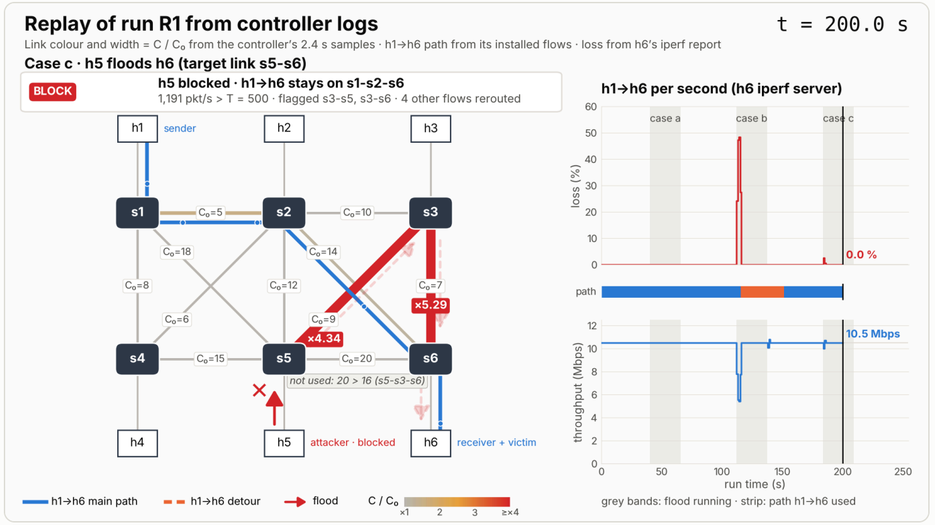
1 60 s, a: s2-s3 marked, path stays



2 114 s, b: before detection

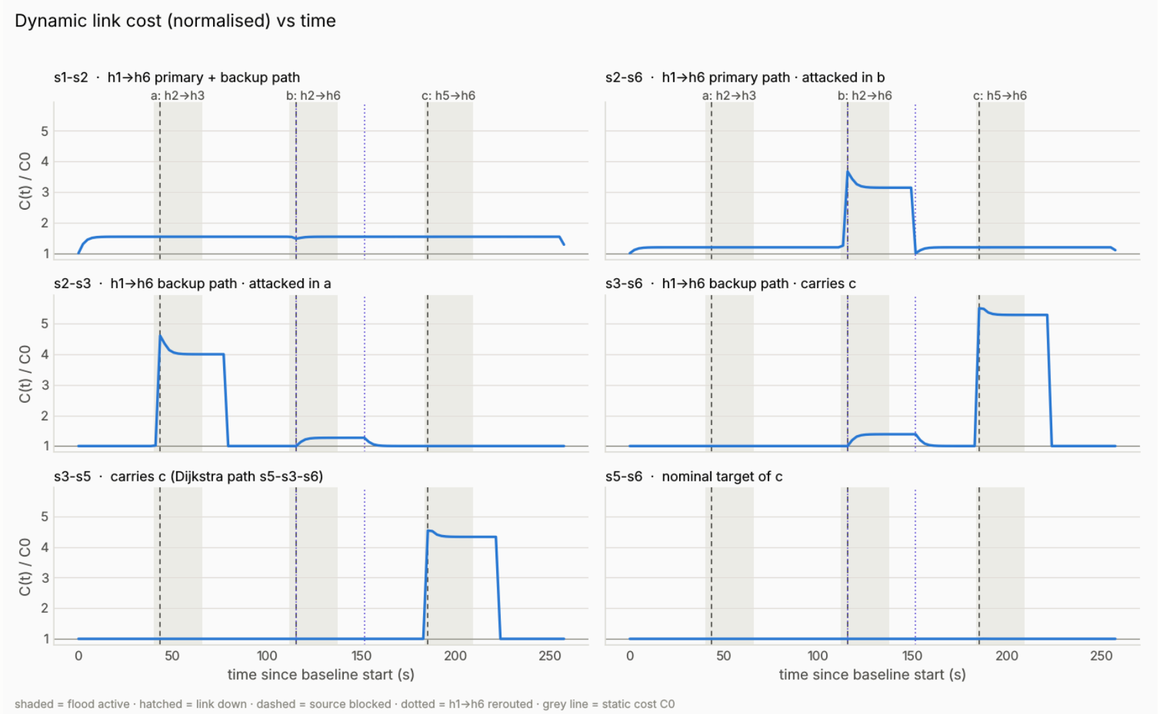


3 135 s: detour, s2-s6 ×3.14

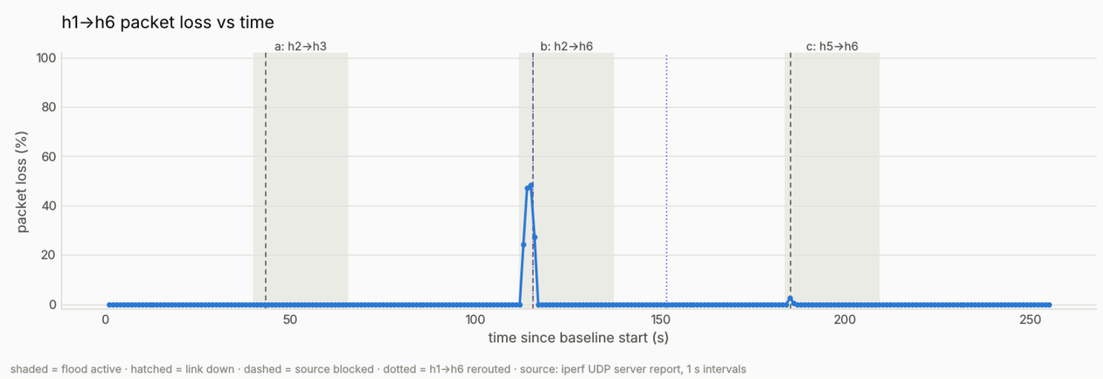


4 200 s, c: s5-s6 stays grey

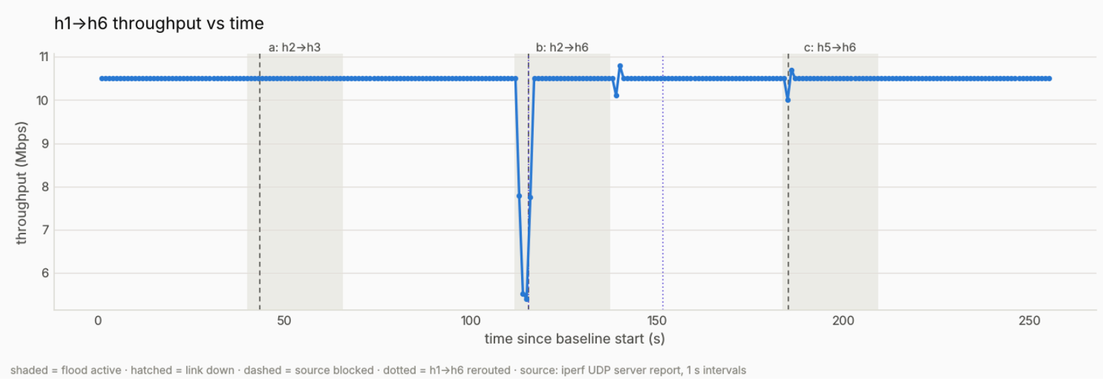
Backup: Task 5 Report Plots (R1), Throughput and RTT



Link cost C/C0 of all monitored links: each attacked link spikes, holds (C0+30)/C0, returns.



h1→h6 loss per second: only before the case b block.



h1→h6 throughput: 10.5 Mbps except the same seconds.

Case b lowest 1 s throughput **5.41 / 4.55 / 6.74 Mbps** (R1 / R2 / R3), otherwise 10.5 Mbps · R1 RTT 0.5 → 20.5 ms before the block: queueing only

Backup: Per-Case Before / During / After (R1, with R2 / R3)

CASE	ATTACK	MARKED, PLATEAU	DETECTION R1/R2/R3	H1→H6 PATH	WORST 1 S LOSS	MEAN LOSS IN FLOOD	LOWEST 1 S MBPS
a	h2→h3 s2-s3	s2-s3 ×4.0	3.4 / 2.2 / 3.5 s	stays s1-s2-s6	0 / 0 / 0%	0%	10.5
b	h2→h6 s2-s6	s2-s6 ×3.14	3.8 / 2.6 / 1.5 s	→ s1-s2-s3-s6 at 115.6 s back at 151.6 s (R1)	48.4 / 56.6 / 34.4%	5.67 / 4.20 / 1.87%	5.41 / 4.55 / 6.74
c	h5→h6 s5-s3-s6	s3-s5 ×4.33 s3-s6 ×5.29	1.7 / 3.0 / 1.8 s	stays s1-s2-s6	2.5 / 4.1 / 6.1%	0.12 / 0.45 / 0.31%	10.0 / 9.88 / 9.65

Before every flood and after recovery: **0% loss, 10.5 Mbps**. Loss in every second ending ≥ 1 s after a block: **0%**.

Attack packets at the victim before the block: **3,608–14,251**; after block + 1 s: **0**. Drop rule on s2 (R1 b): n_packets 31,253.

h1→h6 return in case b: 39.8 / 38.6 / 39.9 s after flood start, i.e. ≈14 s after the flood. Every marked link back to baseline 13.3–15.3 s after the flood.

Stay vs move at the block: 58.78 / 29.97 (R1), 57.22 / 30.10 (R2), 55.34 / 30.22 (R3) · at the return 30.09 vs 24.5.

Backup: Why R1 Case b Took 3.8 s

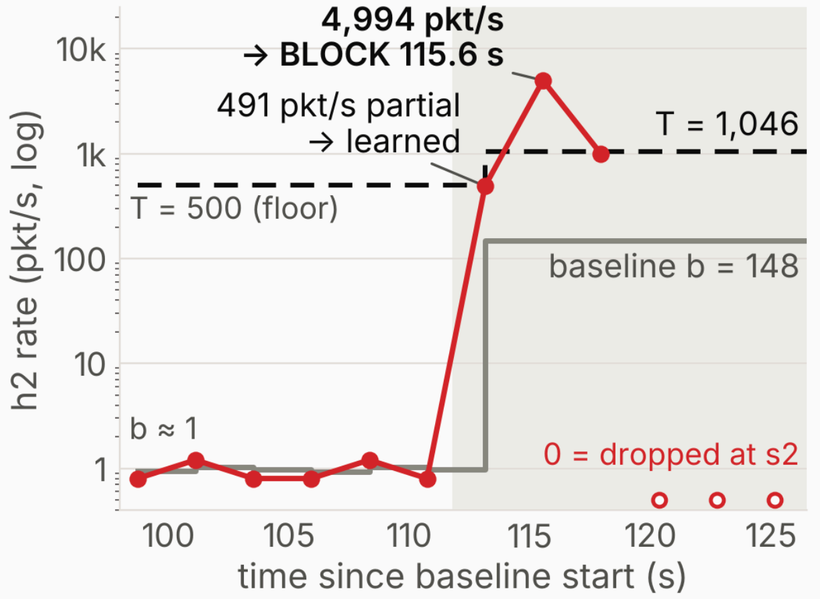
h2 rows of the controller's per-source samples (R1)

T	PKT/S	BASELINE B	THRESHOLD T	VERDICT
110.8 s	–	0.97	500.0	normal
113.2 s	490.7	0.97 → 147.89	500.0 → 1045.61	< 500 → learned
115.6 s	**4993.7**	147.89	1045.61	**> T → BLOCK**

The first sample covered only part of the flood, so it stayed under the floor and was learned.

Fix: cap how far one sample can raise b, or require a full-interval sample.

Detector on h2 during case b (R1)

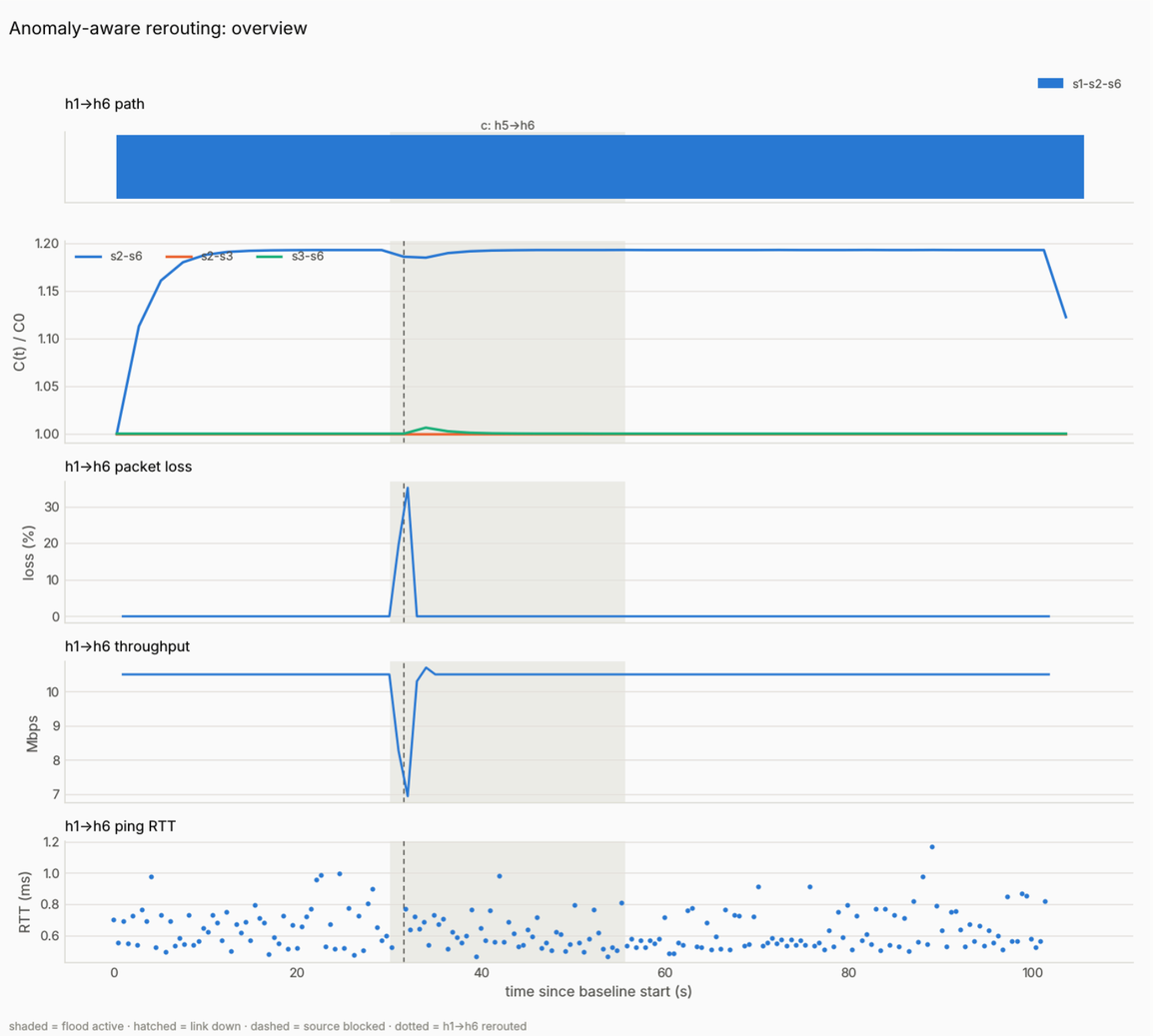


Backup: Case c with s5–s6 Forced (C0 = 8)

```
CEG_COST_OVERRIDE=5-6:8
```

h5→h6 now takes **s5-s6** (8 < 16)

detection	**1.5 s**, only s5-s6 marked
s5-s6 plateau	**(8+30)/8 = 4.75** (4.76 = mean during the flood)
h1→h6	stays on s1-s2-s6
worst 1 s loss	**35%**: h6's access link, the last hop cannot be routed around



Backup: Environment, VM Re-run, Live Demo Commands

	CONTAINER R1–R3	MININET 2.3.0 VM
switch	userspace OVS	kernel OVS, netem delays
detection	1.5–3.8 s	2.2 / 1.6 / 3.8 s
h1→h6 RTT	0.50 ms median	99.8 → 109.0 (detour) → 100.0 ms
stay vs move	58.78 / 29.97 (R1)	54.96 / 30.20
in flight after block	19–101, ≤ 28 ms	487–1,188, ≤ 0.53 s
warm-up	5 samples	12 samples

```
python3 -m unittest discover -s tests   # 18 tests
sudo -E python3 experiments/run_experiment.py   # a,b,c, ~5 min

# live demo, terminal 1
CEG_WARMUP=12 ryu-manager --observe-links \
  --ofp-tcp-listen-port 6633 controller/integrated_controller.py
# terminal 2
sudo -E mn --custom topology/ceg_mesh_topo.py --topo meshTopo \
  --link tc --switch ovsk,protocols=OpenFlow13 --mac \
  --controller remote,ip=127.0.0.1,port=6633
mininet> pingall
mininet> h6 iperf -s -u -i 1 &
mininet> h1 iperf -c 10.0.0.6 -u -b 10M -t 600 &
mininet> h2 hping3 --icmp --flood -d 1000 10.0.0.6 &
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2 | grep 5000
mininet> h6 timeout 3 tcpdump -n -i h6-eth0 icmp and src host 10.0.0.2
```

Backup: Parameters, Design Decisions D1–D13, Code Map

poll 2 s + 0.4 s wait = sample every 2.4 s | detector EMA weight 0.3 | link-load EMA weight 0.6 | T = max(3b, b+4σ, 500) · warm-up 5

α 10 · β 30 · Rmax 40 Mbps | margin 2 · hold 10 s | unblock: drop < 0.5T, 10 s quiet

D	PROBLEM	OUR RULE	CODE (integrated_controller.py unless noted)
D1	Reference cost table is an old 4-switch topology	C0 rewritten for the 11 links (= delay ms)	`sdn_logic.py BASE_COSTS :71`
D2	Rules match eth_dst only: no source rate	match ipv4_src / dst / proto	`flow_stats_reply_handler :374`
D3	Packet counted at every hop	count only at the ingress switch	`:374`
D4	Victim's echo replies look like a flood	skip ICMP echo replies	`:374`
D5	Update then compare: ≈21×, poisoned baseline	compare first, never learn anomalies	`AnomalyDetector.update :207`
D6	Fixed 3× threshold	max(3b, b+4σ, 500) + warm-up	`threshold :194`
D7	After the block R → 0: reroute not held	A = 1 while the drop rule counts, + 10 s	`_evaluate :423`
D8	Attacker's 1 pps ping keeps hitting the rule	still attacking = drop rate ≥ 0.5T	`_evaluate :423`
D9	Own load makes the current path dearer: flapping	what-if + margin 2	`_cost_graph :489 · _reroute_all :505`
D10	Installed rules never change	reinstall egress first, delete stale	`_install_path :327 · _reroute :521`
D11	Broadcast storm in the mesh	ignore packet-in until ready; ARP proxy	`_handle_arp :223 · _flood_to_hosts :249`
D12	Same attacker h2 in a and b	auto-unblock after 10 s quiet	`_block :541 · _unblock :565`
D13	Link-down drops 'ready': ARP ignored	latch ready after first discovery	`_refresh_topology :169`

Contributions and Tools

Who did what

Yang Kaiming A0326309E · system & detection
Topology and MiniEdit, the controller loop, anomaly detection and blocking, the experiment runs and the demo

Chen Danze A0328226E · routing & results
Dynamic link cost, rerouting and recovery, the Task 5 analysis and figures, the extended experiments

Tools

Ryu 4.34 controller, OpenFlow 1.3

Mininet 2.3.0 network emulation, MiniEdit

Open vSwitch switches

iperf legitimate UDP traffic, loss

hping3 ICMP flood

tcpdump victim capture

matplotlib figures

ffmpeg video encoding

QEMU Mininet VM